

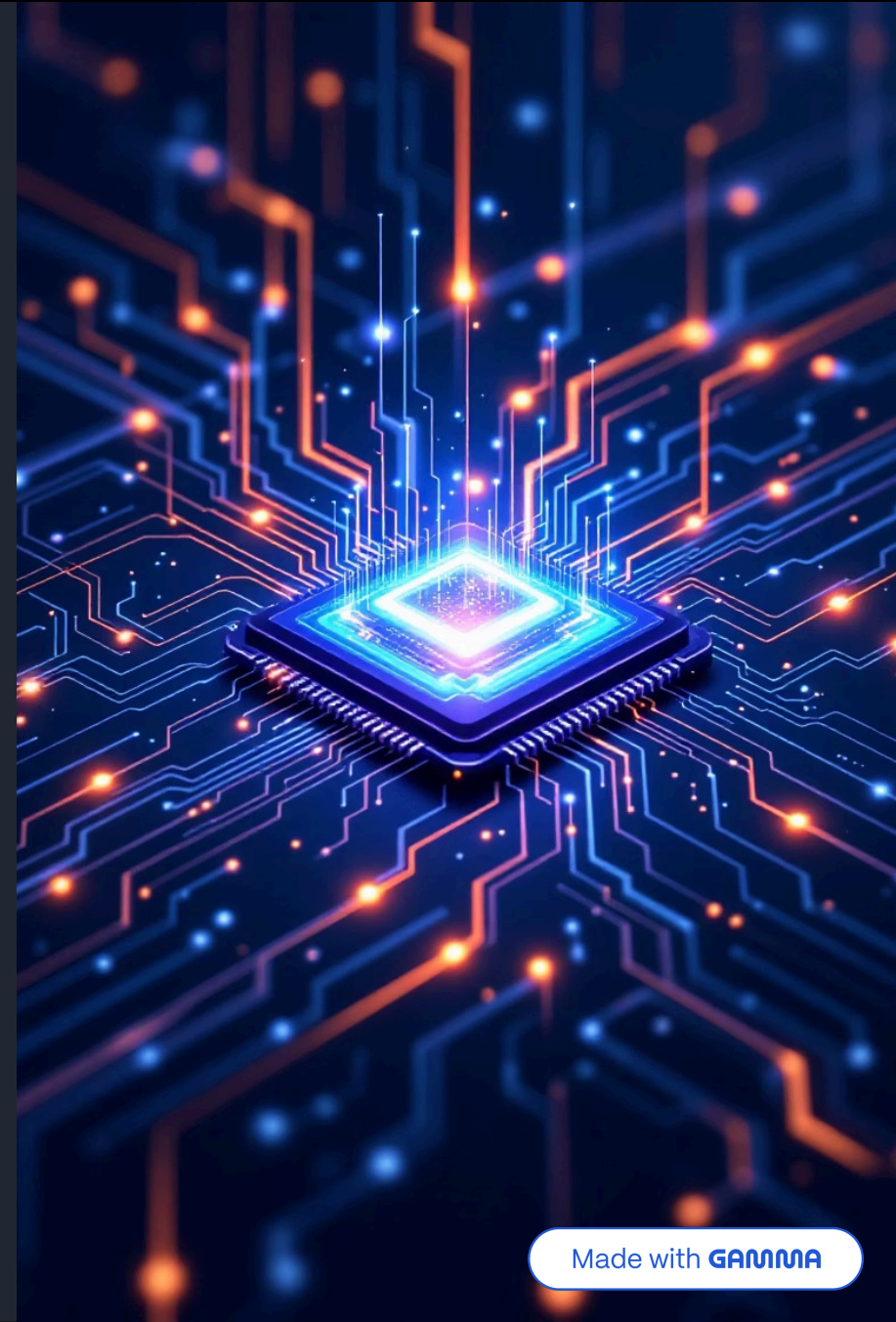
Atividade 3 - Etapa 1: Diagnóstico de DevOps e CI/CD

Auditoria do Projeto Scrapegraph- ai

Disciplina: Engenharia de Software II (UFS)

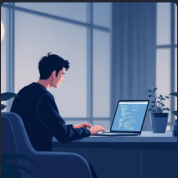
Docente: Prof. Dr. Glauco Carneiro

Data: 30 de Janeiro de 2026



A Equipe e Nossas Responsabilidades

Nossa equipe de especialistas abordará diferentes facetas da auditoria DevOps no projeto Scrapegraph-ai, garantindo uma análise abrangente e profunda.



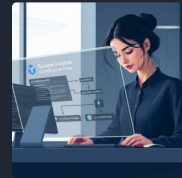
Rafael Santos

Investigação e Coleta de Evidências (Workflows e PRs).



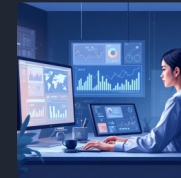
Cauan Machado

Introdução ao Projeto e Relevância.



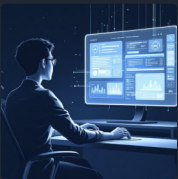
Maria Eduarda

Análise do Fluxo de Contribuição (CONTRIBUTING.md).



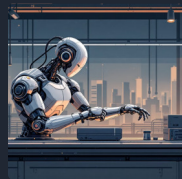
José Gabriel

Workflow de Qualidade (QA e Testes).



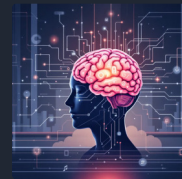
Mateus Barreto

Workflow de Segurança (CodeQL).



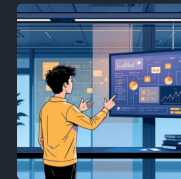
Breno Nascimento

Automação de Release.



José Victor

Ferramentas de Apoio (IA e Copilot).



Pedro Silveira

Demonstração Prática e Conclusão.



Scrapegraph-ai: Uma Ferramenta Inovadora

O Scrapegraph-ai é uma biblioteca Python avançada de web scraping que se destaca por sua capacidade de integrar Modelos de Linguagem Grande (LLMs) e lógica de grafos. Seu principal objetivo é transformar HTML complexo em dados JSON estruturados de forma inteligente e eficiente.

Este repositório open source no GitHub demonstra alta atividade e um compromisso contínuo com a inovação no domínio do web scraping.

Diagnóstico do Cenário Atual (AS-IS)



Maturidade DevOps Elevada

O Scrapegraph-ai demonstra uma maturidade DevOps notável, com uma cultura bem consolidada e processos otimizados. A adoção de práticas de desenvolvimento e operações integradas é evidente.

GitHub Actions: O Núcleo da Automação

A ferramenta principal que impulsiona a automação e os fluxos de trabalho do projeto são os GitHub Actions. Isso garante a execução consistente e confiável dos pipelines.

Automação Completa

O projeto possui pipelines robustos para Testes, Linting, Segurança e Release Automático (Semantic Release). Um diferencial importante é a não-dependência de deploys manuais, evidenciando uma verdadeira integração e entrega contínua.

Evidências Técnicas: Workflows Automatizados

A auditoria detalhada na pasta `.github/workflows` do repositório Scrapegraph-ai revelou a existência de arquivos cruciais para a automação de seus processos.

`code-quality.yml`

Responsável por gerenciar a qualidade do código.

`test-suite.yml`

Define e executa os testes automatizados da aplicação.

`release.yml`

Controla o processo de lançamento automático de novas versões.

A presença desses arquivos comprova o uso de Infraestrutura como Código (IaC), o que é um indicador forte de um ambiente DevOps maduro e bem estruturado.

O Guardião da Qualidade (Quality Gate)

O Scrapegraph-ai implementa um rigoroso "Quality Gate" em seu pipeline, utilizando ferramentas avançadas para garantir a qualidade do código.



Ruff

Ferramenta de linting ultrarrápida, otimizando a verificação de estilo.



Black

Garante a formatação consistente do código Python, eliminando discussões de estilo.



Isort

Organiza automaticamente as importações, mantendo o código limpo e legível.

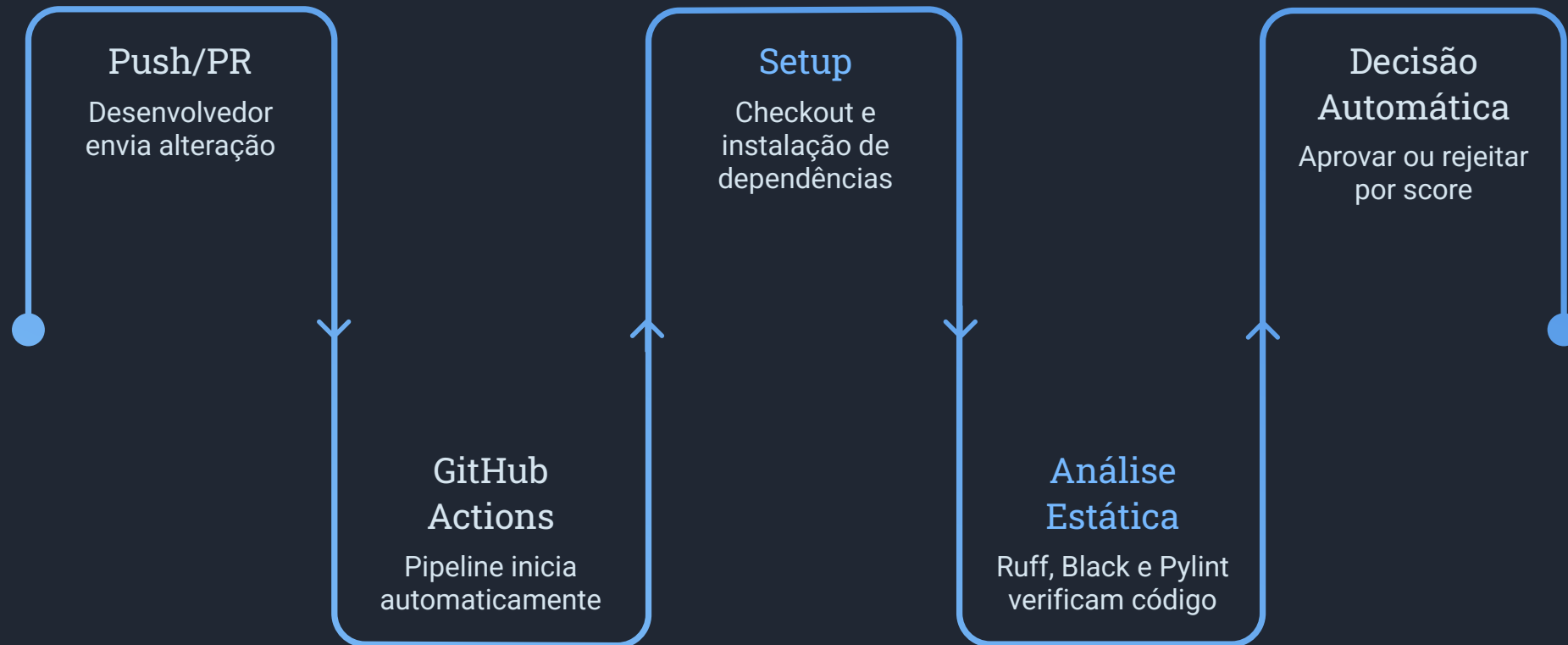


Pylint

Realiza uma análise lógica profunda, detectando erros e problemas de design.

- ❑ **Regra de Bloqueio:** Uma nota do Pylint inferior a 8.0 bloqueia automaticamente o Pull Request, garantindo que apenas código de alta qualidade seja integrado.

Fluxo de Trabalho Mapeado: Do Código ao Release



Este diagrama ilustra a jornada do código desde a contribuição do desenvolvedor até o lançamento automatizado, destacando os pontos de controle e a inteligência integrada no pipeline.

Fluxo de Entrada: Como Contribuir no ScrapeGraphAI

Para garantir uma experiência de contribuição eficiente e alinhada com os padrões do projeto, o Scrapegraph-ai possui um fluxo de entrada bem definido para novos desenvolvedores.



O Guia CONTRIBUTING.md

Comece por este arquivo para entender as diretrizes de contribuição, como o processo de submissão de Pull Requests e o código de conduta.



Estratégia de Branches

Utilize as branches `pre` ou `beta` para seu desenvolvimento. A branch `main` é reservada para releases estáveis.



Ambiente de Desenvolvimento

Instale o gerenciador de pacotes `uv` (em vez do `pip`) para gerenciar as dependências do projeto de forma rápida e eficiente.



Sincronize as Dependências

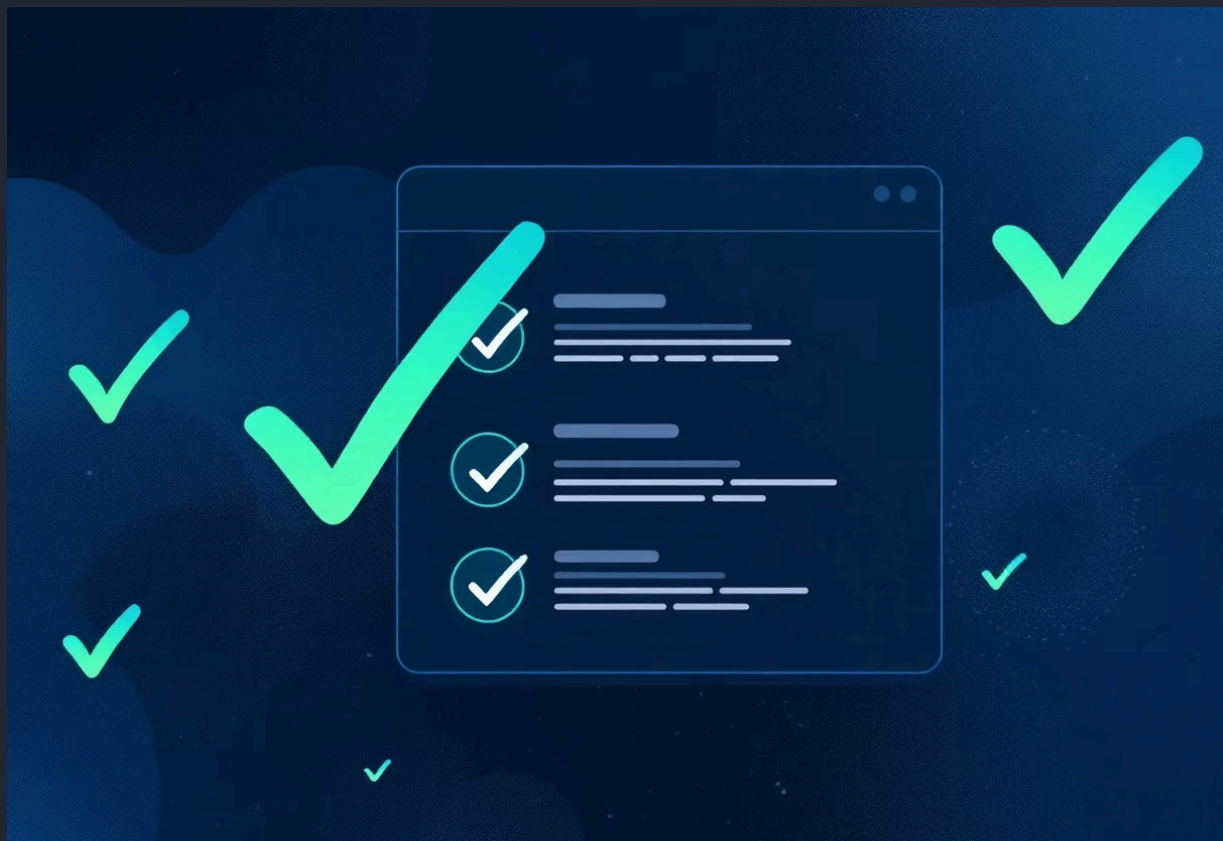
Execute o comando `uv sync` para instalar todas as dependências do projeto e configurar seu ambiente de desenvolvimento.



Hooks de Pré-commit

Os hooks de pré-commit são ativados automaticamente para garantir que seu código siga os padrões de qualidade e estilo antes mesmo de ser comitado.

Execução e Deploy Contínuo



A análise de Pull Requests fechados forneceu evidências claras do sucesso do pipeline CI/CD. Observou-se que todos os merges na branch `main` foram precedidos da confirmação de "7 checks passed".

Este fato crucial demonstra que as etapas de qualidade e segurança foram rigorosamente cumpridas antes que qualquer alteração fosse integrada ao código-fonte principal.

Adicionalmente, após a aprovação e o merge, o bot `github-actions` é acionado para realizar o lançamento da versão de forma totalmente automática, garantindo a entrega contínua do software sem intervenção manual.

Segurança em Primeiro Lugar com CodeQL

O projeto Scrapegraph-ai não se limita à qualidade do código; ele também integra ferramentas avançadas para garantir a segurança. O uso de **CodeQL** é um exemplo claro dessa preocupação.

CodeQL é um poderoso motor de análise de segurança que varre o código em busca de vulnerabilidades e falhas, proporcionando uma camada adicional de proteção contra potenciais ameaças.

Além disso, o **Dependency Review** assegura que todas as dependências utilizadas no projeto sejam seguras e estejam atualizadas, minimizando os riscos associados a vulnerabilidades em bibliotecas de terceiros.



Semantic Release: Automatizando o Lançamento de Versões

O Scrapegraph-ai adota o **Semantic Release** em conjunto com o GitHub Actions para garantir um processo de lançamento de novas versões totalmente automatizado, consistente e livre de erros manuais. Essa prática assegura que cada alteração relevante seja refletida em uma nova versão pública do pacote.



Commits Semânticos

As mensagens de commit seguem um padrão que indica o tipo de alteração (e.g., `feat`, `fix`), guiando o versionamento.



Versão Automática

O `semantic-release` determina e incrementa a versão (major, minor ou patch) com base nos commits semânticos.



Geração de Changelog

Um arquivo de changelog detalhado é gerado automaticamente, registrando todas as modificações da nova versão.

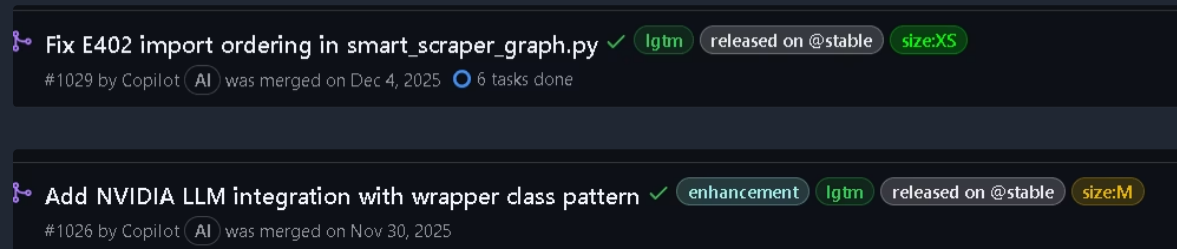


Publicação Automática

Após a geração da versão e do changelog, a nova versão do pacote é automaticamente publicada, garantindo a entrega contínua.

Copilot Coding Agent

O Copilot Coding Agent atua de forma ativa no projeto, escrevendo código e abrindo Pull Requests como um verdadeiro desenvolvedor IA, agilizando o ciclo de desenvolvimento.



Identificação nos Logs

Autor "Copilot [AI]" presente nos históricos de commit e Pull Requests.

Integração NVIDIA LLM

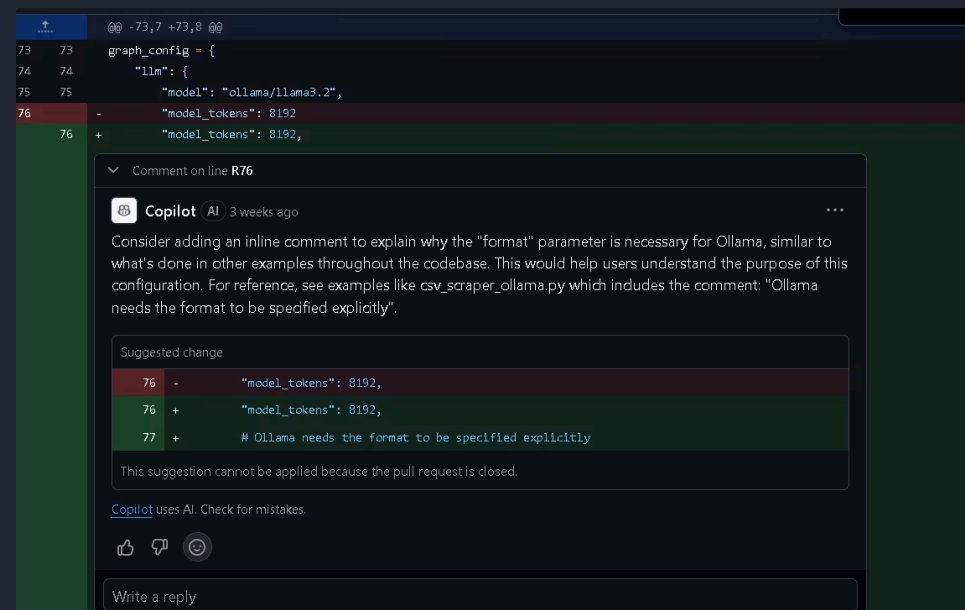
Implementou a integração crucial com NVIDIA LLM, conforme evidenciado no PR #1026.

Manutenções Automáticas

Responsável por correções de formatação e erros de importação (PR #1029).

Copilot Code Review

O Copilot atua como um **auditor sênior** no projeto, revisando códigos já escritos e sugerindo melhorias de forma inteligente e contextualizada.



Sugestões Acionáveis

Propõe mudanças específicas com blocos de código prontos para aplicação, agilizando a implementação das melhorias.

Visão Sistêmica

Demonstra compreensão do contexto do projeto, citando padrões de outros arquivos para embasar suas revisões.

Foco em Clareza

Prioriza a legibilidade e a manutenibilidade, sugerindo a adição de comentários explicativos e aprimoramentos na documentação.

Ajuda ou Ruído? (Análise Crítica)

Ajuda: Impacto Positivo

- **Aumento da Produtividade:** O Copilot remove tarefas repetitivas, como linting e formatação, liberando os desenvolvedores para focar em lógica de negócios.
- **Consistência do Projeto:** Suas revisões são baseadas no histórico do repositório, garantindo padrões de código uniformes e alinhados com o projeto.

Ruído: O Risco Potencial

- **Poluição do Log:** Existe o risco de que muitas notificações ou sugestões irrelevantes pudessem poluir o histórico de commits e Pull Requests.



Precisão no ScrapeGraphAI: No ScrapeGraphAI, observamos que as sugestões do Copilot são **precisas** e **frequentemente aceitas** (Merged), minimizando o "ruído".

Veredito e Próximos Desafios

Após uma auditoria detalhada, o veredito é claro: o projeto Scrapegraph-ai é um exemplo de "Green Build" consistente, com práticas DevOps bem estabelecidas e automação robusta.

1

Build Consistente

A automação e os Quality Gates garantem a estabilidade e confiabilidade.

2

Segurança Reforçada

CodeQL e Dependency Review adicionam uma camada robusta de proteção.

3

Próximo Desafio

Para a Etapa 2, o desafio da equipe será replicar esse pipeline rigoroso no Fork, mantendo a régua de qualidade alta (Pylint Score ≥ 8).

Esta auditoria serve como um modelo para a implementação de práticas de Engenharia de Software de excelência, especialmente no contexto de CI/CD e DevOps.